

StableComposer and GasUSDTokenConverter Audit



December 5, 2025

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	5
Low Severity	7
L-01 Incomplete Docstrings	7
L-02 Missing Docstrings	7
L-03 Missing Zero-Address Checks	8
L-04 EIP-7702 And EIP-4337 Incompatibilities	9
L-05 Tokens May Be Permanently Lost	9
L-06 Unhandled Panic Revert	10
Notes & Additional Information	11
N-01 Magic Numbers	11
N-02 Non-Explicit Imports	11
N-03 Use Custom Error	12
N-04 Unused Named Return Variable	12
N-05 Deviation from EIP-677 Draft Specification	12
Conclusion	14
Appendix	15
Issue Classification	15

Summary

Type	Token	Total Issues	11 (0 resolved)
Timeline	From 2025-11-17 To 2025-11-17	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	6 (0 resolved)
		Notes & Additional Information	5 (0 resolved)

Scope

OpenZeppelin performed an audit of the [Everdawn-Labs/usdt0-oft-contracts](#) repository at commit [22cd088](#).

In scope were the following files:

```
GasUSDT0Converter.sol  
StableComposer.sol
```

System Overview

The `StableComposer` and `GasUSDT0Converter` contracts are a critical part of the Stable network infrastructure. These contracts serve as a primary entry point for USDT0 tokens onto the Stable network. As the network requires USDT0 tokens in order to issue its native gas token, gUSDT, it is crucial that USDT0 bridging and native token issuance execute seamlessly.

The `GasUSDT0Converter` contract exposes deposit and withdrawal operations enabling conversion between native value and USDT0 ERC-20 representations on the Stable network. The minting/burning of native-denominated balances are delegated to the `bank` precompile set upon construction. In addition, the converter integrates directly with the LayerZero OFT contract's `send` function, burning native gas tokens and forwarding cross-chain send requests to the OFT contract.

The `StableComposer` contract serves as the on-chain endpoint for composed OFT messages. When the OFT receives a cross-chain transfer that includes a compose payload, the `lzCompose` function is triggered. `StableComposer` then decodes the payload and utilizes the `GasUSDT0Converter` contract to unwrap the transferred USDT0 into native value. In the event that a payload is malformed, `StableComposer` records the failed transfer and provides a refund mechanism for returning the amount to the source chain via the OFT.

These two contracts work together to allow users to bridge USDT0 tokens onto the Stable network via LayerZero and convert the bridged funds into the native gas token of the Stable network, gUSDT.

Security Model and Trust Assumptions

During the audit, the following trust assumptions were identified:

- The underlying LayerZero `endpoint` and the `OFT` implementation are trusted. `StableComposer` depends on correct `_oApp` configuration and accurate routing by

the `endpoint`. Any misconfiguration can cause message spoofing or incorrect compose delivery.

- The `bank` precompile is assumed to implement secure mint/burn logic. `GasUSDT0Converter` relies on the `bank` to mint and burn native value while maintaining accounting invariants. As such, it is assumed that the contract is granted (and maintains) the requisite `mint` and `burn` privileges.
- No owner or admin role exists within `GasUSDT0Converter` or `StableComposer`, though only the `endpoint` is permitted to call `lzCompose` to deliver message payloads from LayerZero to `StableComposer`.
- It is assumed that the OFT cross-chain transfer flow, which ends with the call to the `StableComposer` contract's `lzCompose` function, is the **only method** by which USDT0 enters the Stable network ecosystem. If USDT0 or gUSDT can be minted on the Stable network by other means, it cannot be guaranteed that the `GasUSDT0Converter` will hold enough USDT0 to cover calls to `depositTo`.

Low Severity

L-01 Incomplete Docstrings

Throughout the codebase, multiple instances of incomplete docstrings were identified:

- In `GasUSDT0Converter.sol`, in the `depositToAndCall` function:
 - the `recipient` and `data` parameters are not documented
 - not all return values are documented
- In `StableComposer.sol`, in the `FailedMessageDecode` event, the `guid`, `sender`, `msgValue`, and `composeMessage` parameters are not documented.

Consider thoroughly documenting all functions/events (and their parameters or return values) that are part of a contract's public API. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec).

Update: Acknowledged, not resolved. The team stated:

| Acknowledged.

L-02 Missing Docstrings

Throughout the codebase, multiple instances of missing docstrings were identified:

- In `GasUSDT0Converter.sol`, the `ITransferReceiver` interface
- In `GasUSDT0Converter.sol`, the `onTokenTransfer` function
- In `GasUSDT0Converter.sol`, the `I677` interface
- In `GasUSDT0Converter.sol`, the `Transfer` event
- In `GasUSDT0Converter.sol`, the `GasUSDT0Converter` contract
- In `GasUSDT0Converter.sol`, the `SendFromWrapper` event
- In `GasUSDT0Converter.sol`, the `Withdraw` event
- In `GasUSDT0Converter.sol`, the `Deposit` event
- In `GasUSDT0Converter.sol`, the `token` state variable
- In `GasUSDT0Converter.sol`, the `oft` state variable
- In `GasUSDT0Converter.sol`, the `bank` state variable
- In `GasUSDT0Converter.sol`, the `receive` function
- In `GasUSDT0Converter.sol`, the `deposit` function

- In `GasUSDT0Converter.sol`, the `depositTo` function
- In `GasUSDT0Converter.sol`, the `withdraw` function
- In `GasUSDT0Converter.sol`, the `withdrawToWithPermit` function
- In `GasUSDT0Converter.sol`, the `withdrawTo` function
- In `StableComposer.sol`, the `RefundSuccessful` event
- In `StableComposer.sol`, the `Unwrap` event
- In `StableComposer.sol`, the `token` state variable
- In `StableComposer.sol`, the `wrapper` state variable
- In `StableComposer.sol`, the `endpoint` state variable
- In `StableComposer.sol`, the `oApp` state variable
- In `StableComposer.sol`, the `failedMessages` state variable

Consider thoroughly documenting all functions (and their parameters) that are part of any contract's public API. Functions implementing sensitive functionality, even if not public, should be clearly documented as well. When writing docstrings, consider following the [Ethereum Natural Specification Format](#) (NatSpec).

Update: Acknowledged, not resolved. The team stated:

| Acknowledged.

L-03 Missing Zero-Address Checks

When operations with `address` parameters are performed, it is crucial to ensure the address is not set to zero. Setting an address to zero is problematic because it has special burn/renounce semantics. Instead, this action should be handled by a separate function to prevent accidental loss of access during value or ownership transfers.

Throughout the codebase, multiple instances of missing zero-address checks were identified:

- The `_token` operation in the `GasUSDT0Converter` contract
- The `_bank` operation in the `GasUSDT0Converter` contract
- The `_refundAddress` operation in the `GasUSDT0Converter` contract
- The `_token` operation in the `StableComposer` contract
- The `_wrapper` operation in the `StableComposer` contract
- The `_endpoint` operation in the `StableComposer` contract
- The `_oApp` operation in the `StableComposer` contract

Consider performing a zero-address check before assigning any value to a state variable.

Update: Acknowledged, not resolved. The team stated:

| Acknowledged.

L-04 EIP-7702 And EIP-4337 Incompatibilities

The Stable network explicitly supports EIP-7702 and EIP-4337. EIP-4337 is a technique that allows smart contracts to behave like user accounts. Under this EIP, there will be EIP-4337 smart contract accounts in addition to Externally Owned Accounts (EOA). Currently, the former account type is incompatible with some code like `tx.origin` because `tx.origin` may be the bundler that initiates the transaction.

EIP-7702 allows an EOA to behave like a contract for a single transaction using specified runtime bytecode. If native tokens are refunded to an EOA caller that has temporary code, it cannot be guaranteed that the transaction will succeed. The `StableComposer` contract's `refundToSrc` function may not be compatible with the aforementioned EIPs as it makes use of `tx.origin` as a potential refund address for excess `msg.value`.

To prevent incompatibility with EIP-4337 and EIP-7702, consider adding a `refundAddress` parameter to the `StableComposer` contract's `refundToSrc` function. As the function is not access-controlled, there is no additional risk in allowing the caller to specify an appropriate refund address.

Update: Acknowledged, not resolved. The team stated:

| Acknowledged.

L-05 Tokens May Be Permanently Lost

The cross-chain flow of the USDT0 token includes two transactions. First, there is a call to the OApp's `lzReceive` function which mints USDT0 to the `StableComposer` contract. This call to `lzReceive` also includes a callback to the LayerZero Endpoint's `sendCompose` function which loads the composed message that will be executed in a separate transaction. In the next transaction, the LayerZero executor calls the `StableComposer` contract's `lzCompose` function to finalize the minting of the native gas token of the Stable network (gUSDT).

This call is the final step in the cross-chain bridging of USDT0 to the Stable network and is the only way for users to create gUSDT. To achieve this, the `StableComposer` contract calls the `GasUSDT0Converter` contract's `withdrawTo` function to mint new gUSDT tokens. The issue is that the user-supplied `_receiver` address may be incapable of receiving native

token transfers (e.g., it may be a contract without a `receive` function) which would cause the `lzCompose` call to fail.

Normally, such a failure results in a reverted transaction with only the loss of the native gas token spent. However, in this case, the USDT0 tokens which were minted to the `StableComposer` contract as part of the `lzReceive` call will remain stuck in it since their transfer is dependent on the receiver address successfully accepting the native gas token during the `lzCompose` call. As there is no other mechanism for the affected user to retry the `lzCompose` call, their funds will remain locked in the `StableComposer` contract.

Consider checking the success of the `GasUSDT0Converter::withdrawTo` call and allowing the control flow to route a failed `withdrawTo` call to `failedMessages`, similar to how a failed decoding is handled within the `lzCompose` function.

Update: Acknowledged, not resolved. The team stated:

| Acknowledged.

L-06 Unhandled Panic Revert

Within the `GasUSDT0Converter` contract's `depositTo` function, there is a call to the `bank` precompile to burn native tokens from the caller. However, at this point in the function, the amount may be `0`, which causes an unhandled revert because the `amount` within the `burn` call must be greater than 0.

Consider verifying that the `amount` requested in `depositTo` is greater than `0` and reverting with an appropriate error should this not be the case.

Update: Acknowledged, not resolved. The team stated:

| Acknowledged.

Notes & Additional Information

N-01 Magic Numbers

Within `GasUSDT0Converter.sol`, an unexplained literal numeric value (i.e., "magic number") of `1e12` is used to facilitate the conversion of USDT0 to gUSDT (and vice versa). This literal value factors in the difference between the gUSDT and USDT0 token decimals and is used:

- to [determine](#) the appropriate amount of gUSDT in exchange for USDT0
- to [determine](#) the appropriate amount of USDT0 in exchange for gUSDT
- to [validate](#) that the amount deposited is a multiple of `1e12`

Consider defining and using a `constant` instead of using the literal value to improve the clarity and maintainability of the codebase.

Update: Acknowledged, not resolved. The team stated:

| [Acknowledged.](#)

N-02 Non-Explicit Imports

The use of non-explicit imports in the codebase can decrease code clarity and may create naming conflicts between locally-defined and imported variables. This is particularly relevant when multiple contracts exist within the same Solidity file or when inheritance chains are long.

Within `StableComposer.sol`, multiple instances of non-explicit imports were identified:

- `import "@openzeppelin/contracts/token/ERC20/IERC20.sol";`
- `import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";`

Following the principle that clearer code is better code, consider using the named import syntax (`import {A, B, C} from "X"`) to explicitly declare which contracts are being imported.

Update: Acknowledged, not resolved. The team stated:

| Acknowledged.

N-03 Use Custom Error

Since Solidity version 0.8.4, custom errors provide a cleaner and more cost-efficient way to explain to users why an operation failed.

The `GasUSDT0Converter` contract has a `require` statement.

For conciseness and gas savings, consider replacing `require` statements with custom errors.

Update: Acknowledged, not resolved. The team stated:

| Acknowledged.

N-04 Unused Named Return Variable

Named return variables are a way to declare variables that are meant to be used within a function's body for the purpose of being returned as that function's output. They are an alternative to explicit in-line `return` statements.

In `GasUSDT0Converter.sol`, the `success` return variable of the `depositToAndCall` function is unused.

Consider either using or removing any unused named return variables.

Update: Acknowledged, not resolved. The team stated:

| Acknowledged.

N-05 Deviation from EIP-677 Draft Specification

The `GasUSDT0Converter` contract exposes `depositToAndCall`, which first executes `depositTo` to transfer `token` to `recipient` and then emits the `I677.Transfer` event before invoking `ITransferReceiver(recipient).onTokenTransfer`. The ERC-20 `Transfer` event associated with the actual token movement is emitted by the `token` contract during `safeTransfer`, while the EIP-677-style `I677.Transfer` event is emitted by the wrapper.

The `GasUSDT0Converter` contract explicitly notes that the `depositToAndCall` function intends to follow the ERC-677 pattern. However, EIP-677 is in draft stage and is not an officially recognized standard. Furthermore, the observed behavior deviates from EIP-677 expectations in two ways. First, the `I677.Transfer` event is emitted by `GasUSDT0Converter`, so the log's address is the wrapper and not the `token`. Indexers and tools that associate ERC-677 `Transfer` events with the token address will not attribute this event to the `token`, leading to off-chain inconsistencies. Second, the callback is executed by the wrapper, making `msg.sender` in `onTokenTransfer` equal to `GasUSDT0Converter` instead of the `token`.

Consider explicitly documenting the fact that the `depositToAndCall` function mimics the `transferAndCall` function of EIP-677, but warn integrators that the EIP-677 specification is not being implemented. In addition, consider removing or renaming the `I677.Transfer` emission in `GasUSDT0Converter` to avoid signaling EIP-677 compliance, and document the fact that, in callbacks, `msg.sender` is the `GasUSDT0Converter` contract.

Update: *Acknowledged, not resolved. The team stated:*

| *Acknowledged.*

Conclusion

The `StableComposer` and `GasUSDT0Converter` contracts are critical pieces of infrastructure that enable USDT0 tokens to be transferred to and utilized within the Stable network as native gas tokens.

While no high-severity issues were identified, several low-severity issues and notes have been reported. In particular, it was noted that USDT0 tokens may become stuck in an edge case whereby the recipient of a transfer is a contract that does not implement a `receive` function. In addition, the trust assumptions note that the minting of gUSDT tokens should go hand-in-hand with USDT0 receipts. While the Everdawn team has acknowledged these lower-severity issues, no changes are to be made at this time. It is recommended that the issues highlighted be well documented as potential pitfalls for integrators.

Overall, the codebase was found to be clear and well-structured. The Everdawn team is appreciated for their assistance during the audit process.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.